講義 15：Function Module（放在系統裡、任何程式都能呼叫的共用函式）——SE37 與 CALL FUNCTION（授課順序：接在講義 8 之後）

> 對應練習：ex15 | 答案物件：`ZFG_TR15` / `Z_TR15_CALC_REVENUE` / `ZR_TR15_CALL_FM`

本講重點

- Function Module（FM）的定位：**跨程式共用**的邏輯單位
- Function Group：FM 的容器——**一個 group 可以放多個 FM**
- **TOP include 的全域變數：同一個 group 的 FM 共用同一份資料**（呼叫順序、生命週期、陷阱）
- 介面四區：IMPORTING / EXPORTING / CHANGING / TABLES ＋ EXCEPTIONS
- **傳整張表：用 `CHANGING` + DDIC Table Type 取代舊式 `TABLES`**（介面參數一定要用 DDIC 型別）
- SE37 建立與單獨測試
- `CALL FUNCTION` 呼叫：方向對應與例外處理
- FM vs FORM 的選擇

1. FM 是什麼、跟 FORM 差在哪

FORM 只能在同一支程式（含 INCLUDE）裡呼叫；FM 是**全系統共用**的邏輯單位——任何程式都能 `CALL FUNCTION` 呼叫，還能在 SE37 **單獨測試**（不用寫測試程式）。講義 8 的 FORM 是「同一支程式內」的模組化（Modularization），FM 則把同樣的想法擴大到「全系統」。SAP 本身就提供大量標準 FM，之後講義 9 的 ALV（`REUSE_ALV_GRID_DISPLAY`）就是其中一支——學完本講，你呼叫它時就能讀懂每一段參數。

	FORM	**Function Module**
共用範圍	本程式內	全系統
單獨測試	不行	SE37 直接測
介面	USING / CHANGING	IMPORTING / EXPORTING / CHANGING / TABLES + EXCEPTIONS
容器	報表程式	Function Group
定位	程式內部拆段	跨程式共用、RFC、BAPI 的基礎

2. Function Group：FM 的容器

FM 不能單獨存在，必須掛在 **Function Group**（SE37 → Goto → Function Groups → Create，或 SE80）底下。命名 `ZFG_xxx`，同一個 group 的 FM 共用全域資料與 subroutine——把同主題的 FM 收在同一個 group（如 `ZFG_TR15` 放訓練用計算類 FM）。

2.1 一個 Function Group 可以放多個 FM

Function Group 本質上是**一支特殊的程式**（`ZFG_TR15` 對應程式 `SAPLZFG_TR15`），底下用一組 include 組織起來，SE80 展開 Function Group 就看得到。include 是「被主程式原地展開的程式碼片段」，本身不能執行、也

沒有自己的變數空間，可以先把它想成「同一支程式被拆成好幾個檔案」；自己的報表怎麼這樣拆，講義 14 會細講。

Include	內容
`LZFG_TR15TOP`	**TOP include：全域宣告**（`FUNCTION-POOL` 陳述式、全域 `DATA` / `TYPES`），本節重點
`LZFG_TR15UXX`	系統自動維護，列出這個 group 的所有 FM include
`LZFG_TR15U01`、`U02`……	**每個 FM 各一個 include**，放 `FUNCTION ... ENDFUNCTION` 本體
`LZFG_TR15F01`（選用）	同一個 group 內 FM 共用的 FORM

- **一個 FM 只屬於一個 Function Group，一個 group 可以有很多 FM**：在 SE37 對同一個 group 一支一支新增即可，Part 1 的 `Z_TR15_CALC_REVENUE`、Part 4 的 `Z_TR15_CALC_REVENUE_TAB` 就已經同屬 `ZFG_TR15`。
- **分組原則**：要共用同一份資料、同一批共用 FORM 的 FM 放同一個 group；彼此沒有任何共用的 FM 不必硬塞在一起——group 越大，第一次呼叫時要載入的程式越大。
- 呼叫端**不需要**知道 FM 在哪個 group：`CALL FUNCTION 'Z_TR15_...'` 只認 FM 名稱。
- **跟 TR 的關係（講義 8a）**：Function Group 建在正式 Package 時，整組在 TR 裡記錄成一筆 `R3TR FUGR <group 名稱>`（課堂實例 `ZB_FG01`、`ZS_FG01`），裡面的 TOP、UXX、每支 FM 的 include 會跟著一起傳輸——所以同一個 group 的 FM 是綁在一起走的，這也是「彼此無關的 FM 不要硬塞同一個 group」的另一個理由。

2.2 TOP include 的全域變數：同一個 group 的 FM 共用同一份

寫在 TOP include 的 `DATA`，是**整個 Function Group 共用的全域變數**——group 裡每一支 FM 看到、改到的都是同一份。範例（練習 15 Part 5）：

```
* LZFG_TR15TOP（TOP include）
FUNCTION-POOL zfg_tr15.

DATA: gv_total_revenue TYPE s_price,        " 累計營收
      gv_call_count    TYPE i.              " 累加了幾次
```

```
FUNCTION z_tr15_add_revenue                  " FM 1：累加
  IMPORTING VALUE(iv_revenue) TYPE s_price.
  gv_total_revenue = gv_total_revenue + iv_revenue.
  gv_call_count    = gv_call_count + 1.
ENDFUNCTION.

FUNCTION z_tr15_get_total                    " FM 2：讀取（自己沒有累加）
  EXPORTING VALUE(ev_total) TYPE s_price
            VALUE(ev_count) TYPE i.
  ev_total = gv_total_revenue.
```

```
  ev_count = gv_call_count.
ENDFUNCTION.

FUNCTION z_tr15_reset_total.                    " FM 3：歸零
  CLEAR: gv_total_revenue, gv_call_count.
ENDFUNCTION.
```

呼叫端：

```
CALL FUNCTION 'Z_TR15_ADD_REVENUE' EXPORTING iv_revenue = '50000.00'.
CALL FUNCTION 'Z_TR15_ADD_REVENUE' EXPORTING iv_revenue = '40000.00'.
CALL FUNCTION 'Z_TR15_ADD_REVENUE' EXPORTING iv_revenue = '36000.00'.

CALL FUNCTION 'Z_TR15_GET_TOTAL'
  IMPORTING ev_total = gv_total ev_count = gv_count.
WRITE: / gv_total, gv_count.                    " 126,000.00  3
```

重點：三次 `ADD_REVENUE` 只傳「這一筆」，**沒有任何變數帶回呼叫端**；換另一支 `GET_TOTAL` 卻讀得到累加結果——因為兩支 FM 同屬 `ZFG_TR15`，操作的是同一份 `gv_total_revenue`。（實測輸出：第一次呼叫前 `0.00 / 0`；累加三筆後 `126,000.00 / 3`；`RESET_TOTAL` 之後又回到 `0.00 / 0`。）

2.3 生命週期與注意事項

- **載入時機**：Function Group 在**第一次呼叫其中任何一支 FM 時**整個載入記憶體（連同 TOP include 的全域變數，從初始值開始）；之後同一個 session 內一直保留，所以 FM「有記憶」。
- **範圍是「這次 session 的這個 group」**：程式結束、session 結束就消失；不同使用者、不同 session 各自一份，互不影響；**不同 Function Group 的 FM 看不到彼此的全域變數**（要傳資料就用參數）。
- **呼叫順序變成隱藏的前提**：先 `GET_TOTAL` 再 `ADD_REVENUE`，跟先 `ADD_REVENUE` 再 `GET_TOTAL`，結果不同——FM 的介面看不出這個依賴，維護時最容易踩。用前先 `RESET` 是良好習慣。
- **難測、難追蹤**：FM 的行為不只取決於參數，還取決於「之前發生過什麼」；ABAP Unit 測試、除錯都比較麻煩。
- **TOP include 的宣告能用在 FM「本體」裡，但不能用在 FM「介面」型別上**：FM 內可以自由讀寫 `gv_...`、使用 TOP 宣告的本地 `TYPES` 來宣告區域變數；但 IMPORTING / EXPORTING / CHANGING 參數的型別必須是 DDIC 型別（見 3.1 與 Part 4）。
- **定位**：這是傳統 FM 的能力，舊程式常見（把中間結果暫存在 group 裡，配合「先設定、後讀取」的呼叫順序）；新世代開發用 Class 的 instance 屬性——狀態明確、可以建立多份互不干擾，OOP 課程會再比較。

3. 定義介面（SE37 分頁）

範例：計算航班營收的 FM `Z_TR15_CALC_REVENUE`：

```
FUNCTION z_tr15_calc_revenue
  IMPORTING
    VALUE(iv_price)    TYPE s_price        " 收進來：票價
    VALUE(iv_seatsocc) TYPE s_seatsocc     " 收進來：已售座位
```

```
  EXPORTING
    VALUE(ev_revenue)  TYPE s_price       " 送出去：營收
  EXCEPTIONS
    invalid_input.                        " 錯誤情況丟給呼叫端

  IF iv_price < 0 OR iv_seatsocc < 0.
    RAISE invalid_input.                  " 丟例外：中止 FM，呼叫端 sy-subrc <> 0
  ENDIF.

  ev_revenue = iv_price * iv_seatsocc.

ENDFUNCTION.
```

介面四區 + 例外：

區	方向	慣例前綴
IMPORTING	呼叫端 → FM（輸入）	`iv_` / `is_` / `it_`
EXPORTING	FM → 呼叫端（輸出）	`ev_` / `es_` / `et_`
CHANGING	雙向	`cv_` / `cs_` / `ct_`
TABLES	內表（Internal Table）（**官方標記 obsolete**，維護舊 FM 會遇到；新介面用 CHANGING 或 IMPORTING/EXPORTING 傳表格型別（Table Type），原因見下方 3.1 節）	`t_`
EXCEPTIONS	具名的錯誤情況，用 `RAISE 名稱.` 觸發	小寫底線命名

參數型別**一定要用 DDIC 型別**（如 `s_price`、講義 25 建的 Table Type），不能用程式裡自己宣告的 `TYPES`——FM 是跨程式介面，型別要全系統看得到，語意也才一致。

3.1 為什麼 `TABLES` 是「舊式」：官方文件的棄用理由

`TABLES` 參數在 SAP 官方 ABAP Keyword Documentation 裡被明確標記為 obsolete（`ABENFUNCTION_MODULES_OBSOLETE` / `ABAPTABLES_PARAMETERS_OBSOLETE`），原文寫著：「Table parameters are obsolete `CHANGING` parameters that are typed as internal standard tables with a header line.」——`TABLES` 本質上就是舊式的 `CHANGING` 參數，只是型別被限定成「帶 Header Line 的內部表」（Header Line 是什麼、為什麼有問題，見講義 5 第 8 節）。

拼合官方文件多處說明，具體原因有四個：

1. **繼承 Header Line 的雙義問題**：`TABLES` 參數在 FM 內部會自動生成一個同名 work area，容易搞混存取的是整張表還是單筆。
2. **只能傳址，不能傳值**：官方文件明講「Pass by value is not possible in formal parameters defined using `TABLES`」——限制比 `CHANGING` / `IMPORTING` / `EXPORTING` 都嚴格。
3. **型別受限**：只能是 DDIC Table Type（flat line type 的 Standard Table）或泛型 `STANDARD TABLE`，不能用 Sorted/Hashed Table 或深層結構（Deep Structure）。
4. **Class 裡完全禁用**：這也是為什麼 Method（OOP 課程會學到）從來沒有 `TABLES` 這個選項——OO ABAP 設計時直接排除了整套有問題的機制，只留 `IMPORTING`/`EXPORTING`/`CHANGING`/`RETURNING`。

新 FM 的建議寫法：把 `TABLES it_xxx` 換成 `CHANGING it_xxx TYPE <table_type>`；如果表格資料其實是單向的（只進不出、或只出不進），用 `IMPORTING` / `EXPORTING` 搭配表格型別更精確，比一律用 `CHANGING` 更清楚表達方向。

⚠️ **例外**：Remote-enabled FM（RFC / BAPI）如果 RFC log 沒設成 basXML，`TABLES` 傳輸實際上比 `CHANGING` **明顯更快**——這是官方文件列出的唯一還留著 `TABLES` 的實務理由，也是為什麼很多老 BAPI（如 `BAPI_*`）至今介面上還看得到 `TABLES`，不是設計不良，是效能考量下刻意保留。

3.2 新寫法：`CHANGING` + DDIC Table Type 取代 `TABLES`

以練習 15 Part 4 的 `Z_TR15_CALC_REVENUE_TAB` 為例：傳進一整張航班表，FM 逐列算出營收、直接寫回原表。

先在 SE11 準備型別（建法見講義 25 第 6.6 節）：

物件	名稱	內容
Structure（每一列的長相）	`ZTR15_FLIGHT_REV`	`CARRID` / `CONNID` / `PRICE` / `SEATSOCC` / `REVENUE` / `CURRENCY`，欄位都引用標準 Data Element（如 `S_CARR_ID`、`S_PRICE`）
Table Type	`ZTR15_TT_FLIGHT_REV`	Line Type = `ZTR15_FLIGHT_REV`，Standard Table

> `PRICE`、`REVENUE` 是金額欄位，Structure 裡要有 `CURRENCY` 欄位當參考幣別，否則啟用報「specify reference table AND reference field」。

新寫法（本課程的答案，已在 SAP 驗證）：

```
FUNCTION z_tr15_calc_revenue_tab
  CHANGING
    VALUE(ct_flights) TYPE ztr15_tt_flight_rev.       " DDIC Table Type

  FIELD-SYMBOLS <fs_flight> TYPE ztr15_flight_rev.    " 指向一列（講義 16）

  LOOP AT ct_flights ASSIGNING <fs_flight>.
    <fs_flight>-revenue = <fs_flight>-price * <fs_flight>-seatsocc.
  ENDLOOP.

ENDFUNCTION.
```

呼叫端：

```
DATA: gt_flights TYPE ztr15_tt_flight_rev,            " 同一個 Table Type
      gs_flight  TYPE ztr15_flight_rev.

CLEAR gs_flight.
gs_flight-carrid   = 'LH'.
gs_flight-connid   = '0400'.
gs_flight-price    = '500.00'.
gs_flight-seatsocc = 100.
gs_flight-currency = 'EUR'.
APPEND gs_flight TO gt_flights.
* ……再 APPEND 幾筆，REVENUE 先留空

CALL FUNCTION 'Z_TR15_CALC_REVENUE_TAB'
  CHANGING
    ct_flights = gt_flights.                          " 呼叫後 REVENUE 已被填上
```

對照舊式 `TABLES` 寫法（維護舊 FM 會遇到，看得懂即可）：

```
FUNCTION z_old_calc_revenue
  TABLES
    t_flights STRUCTURE ztr15_flight_rev.             " 帶 Header Line 的內表

  LOOP AT t_flights.                                  " 沒有 INTO：用 Header Line
    t_flights-revenue = t_flights-price * t_flights-seatsocc.
    MODIFY t_flights.                                 " 改的是 Header Line，要 MODIFY
寫回
  ENDLOOP.

ENDFUNCTION.
```

	舊式 `TABLES`	**新式 `CHANGING` + Table Type**
參數型別	`STRUCTURE` 結構，自動帶 Header Line	DDIC Table Type，沒有 Header Line
`t_flights` 指的是	整張表還是那一列？要看語境（雙義）	永遠是整張表，一列用 work area 或 Field-Symbol
傳遞方式	只能傳址	可以傳值（`VALUE(...)`）或傳址
Method（OOP）能不能用	不能	能——同一套寫法直接沿用到 Class

為什麼型別一定要是 DDIC Table Type：FM 的介面要讓**任何呼叫端**都能獨立做語法檢查，不能依賴 Function Group 裡自己宣告的 `TYPES`（第 2.3 節提過，那只能用在 FM 本體）。所以要傳表格，就得先在 SE11 建好

Table Type——這也是講義 25 排在本講之前的原因。

方向要選對：資料有進有出（本例：傳進航班、補上營收再傳回）用 `CHANGING`；只進不出用 `IMPORTING`、只出不進用 `EXPORTING`，一樣搭配 Table Type，介面比一律用 `CHANGING` 更清楚。

4. SE37 單獨測試

SE37 → 輸入 FM 名 → Test（F8）→ 填 IMPORTING 值 → 執行，直接看 EXPORTING 結果與例外。**先在 SE37 測通，再寫呼叫端**——問題切成兩半，好查十倍。測試案例還能存起來（Test Data Directory）重複使用。

5. CALL FUNCTION：呼叫端

```
DATA gv_revenue TYPE s_price.

CALL FUNCTION 'Z_TR15_CALC_REVENUE'
  EXPORTING
    iv_price      = '1500.00'
    iv_seatsocc   = 200
  IMPORTING
    ev_revenue    = gv_revenue
  EXCEPTIONS
    invalid_input = 1
    OTHERS        = 2.
IF sy-subrc = 0.
  WRITE: / '營收 =', gv_revenue.
ELSE.
  WRITE: / '呼叫失敗，sy-subrc =', sy-subrc.
ENDIF.
```

兩個最容易搞混的點：

方向反轉——區段名稱是站在「自己」的立場寫的：

- FM 定義的 IMPORTING（它要**收**的）→ 呼叫端寫在 **EXPORTING**（我要**送**的）
- FM 定義的 EXPORTING（它要**給**的）→ 呼叫端寫在 **IMPORTING**（我要**收**的）

EXCEPTIONS 的數字——「發生該例外時，我的 sy-subrc 要變成幾」：FM 裡 `RAISE invalid_input.` → 呼叫端 sy-subrc = 1；`OTHERS = 2` 接住沒列名的例外。**沒列 EXCEPTIONS 又沒 OTHERS 時，例外直接讓程式 dump**——CALL FUNCTION 之後檢查 sy-subrc 跟 READ TABLE 之後一樣是鐵律。

- FM 名稱是**字串**（大寫、加引號）；參數名不加引號。
- 常數（Constant）/字面值（Literal）可以直接餵 EXPORTING（如 `'1500.00'`），IMPORTING 必須接變數。

6. 什麼時候用 FM

- 邏輯要給**多支程式**共用 → FM（或 OOP 課程後：Class Method）。
- 只是本程式內拆段 → FORM 就好，不必為拆而拆 FM。

- FM 還是 RFC（跨系統呼叫）與 BAPI 的技術基礎——維護介面程式一定會遇到。
- 開發前先搜尋：標準 FM 幾千支，常見需求（日期換算、單位轉換、彈窗）多半已有現成的。

7. 常見錯誤與陷阱

症狀	原因
呼叫端寫 `iv_price` 在 IMPORTING 下報錯	方向搞反：FM 的 IMPORTING 要寫在呼叫端 EXPORTING
執行到一半 dump（RAISE_EXCEPTION）	FM RAISE 了例外，呼叫端沒列 EXCEPTIONS 承接
sy-subrc = 2 查不出原因	落進 OTHERS——把具名例外逐一列出對應數字好定位
FM 名對了卻說不存在	呼叫字串沒大寫、或 FM 沒啟用
SE37 測試正常、程式呼叫結果不同	呼叫端參數對應錯（依名稱對應，檢查每個等號左邊）
兩支 FM 讀不到對方設的全域變數	它們不在同一個 Function Group（全域變數只在 group 內共用），或是不同 session
同一支程式重複呼叫累加類 FM，結果越來越大	全域變數在 session 內一直保留，沒有先 RESET
改了 FM 呼叫順序，結果就不一樣	FM 依賴 group 全域變數的狀態，順序成了隱藏前提——文件要寫清楚，或改用參數傳遞

8. 課堂練習

完成 ex15：建 `ZFG_TR15` 與營收計算 FM（含防呆例外）、SE37 單測通過，再寫報表呼叫並驗證正常/例外兩條路；Part 5 在同一個 Function Group 加三支 FM，透過 TOP include 的全域變數累加與讀取。